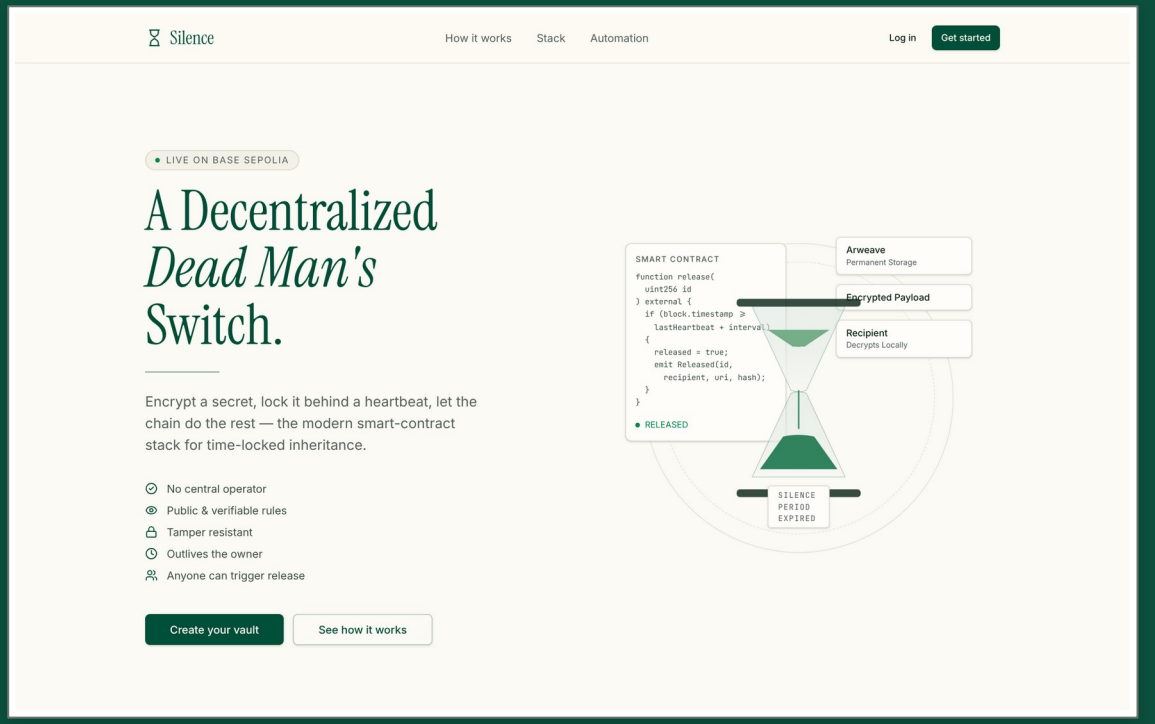
PROJECT DEEP-DIVE · ARCHITECTURE

# Silence

## *A Decentralized Dead Man's Switch*

Client-side encryption · IPFS · Solidity on Sepolia · Chainlink Automation




React 19 · ethers v6 · Supabase · Pinata · Vercel

# Secrets need a plan for when their owner goes silent



### Keys die with their holder

Seed phrases, passwords and final letters are lost if nobody can reach them after an accident or death.



### Human executors are a single point of failure

A lawyer, friend or server can forget, refuse, be compromised, or simply go offline.



### Silence itself should be the trigger

If the owner stops proving they are alive, release should happen automatically and verifiably.

**Silence's answer:** lock an encrypted secret and ETH behind a heartbeat. Stop pinging, and the blockchain itself releases the funds and makes the ciphertext claimable. No operator can block or rush it.

# What this deck covers

01

## System architecture

Components, tech stack, trust boundaries, module map

02

## Smart contract

Data model, state machine, Chainlink Automation, safety patterns

03

## Cryptography & storage

PBKDF2 + AES-256-GCM pipeline, IPFS, Supabase index

04

## User journeys

Every screen, from sign-in to the beneficiary decrypting

05

## Review & roadmap

Strengths, known gaps, deployment, next steps

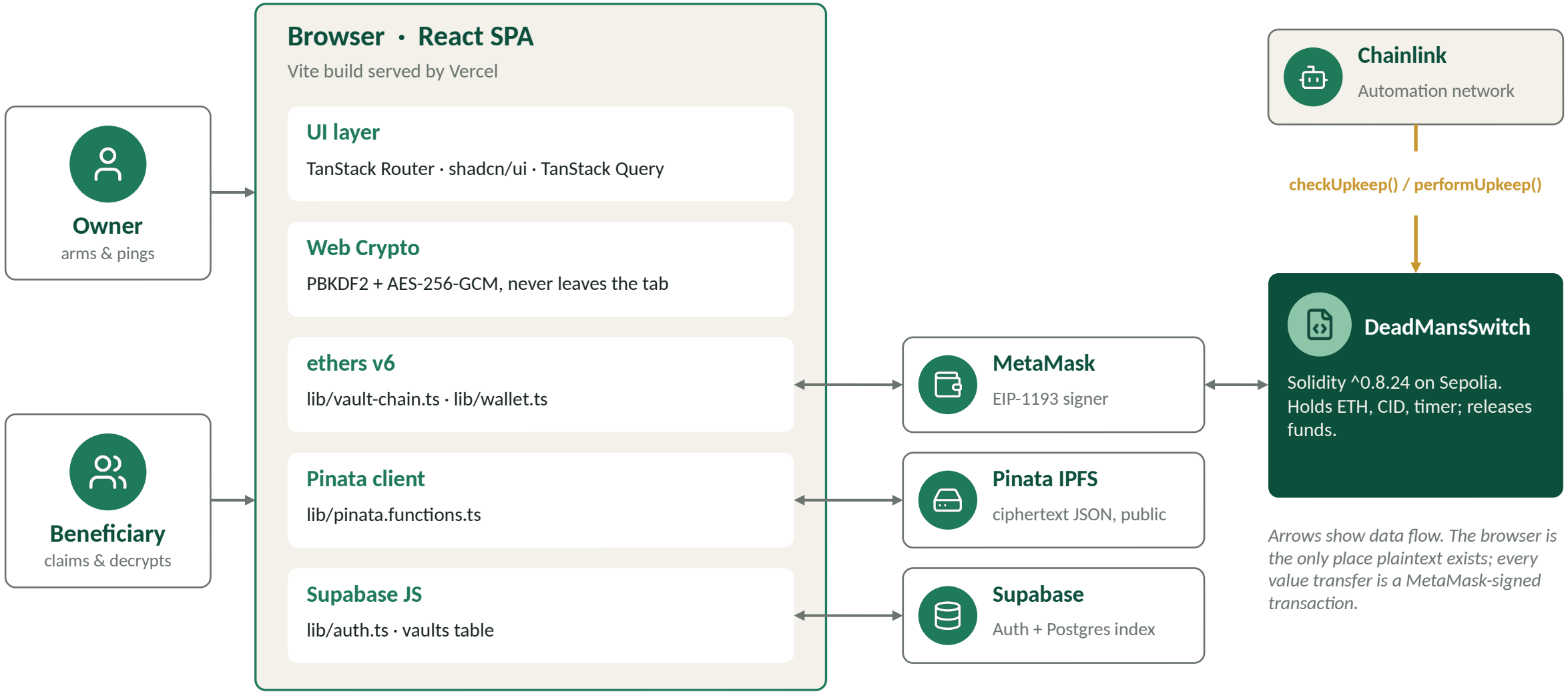
Screenshots were captured from a local build of this repository with demo data (mocked wallet, Supabase and IPFS responses).

01

# System architecture

Five moving parts: the browser, the wallet, the contract, IPFS and Supabase, plus an automation network that never needs trusting.

# High-level architecture




*Arrows show data flow. The browser is the only place plaintext exists; every value transfer is a MetaMask-signed transaction.*

# Layered tech stack

| Layer | Technology | Where in the code | Responsibility |
|---|---|---|---|
| **Presentation** | React 19, TanStack Router, Tailwind v4, shadcn/ui (Radix), framer-motion | `src/routes, src/components/vault` | Landing, auth, owner & beneficiary views |
| **Data fetching** | TanStack Query with 10–15 s polling | `Dashboard.tsx, BeneficiaryPanel.tsx` | Keeps on-chain state live |
| **Identity** | Supabase Auth: email/password, Google OAuth | `lib/auth.ts, routes/auth.tsx` | Sessions, profiles, roles |
| **Cryptography** | Web Crypto: PBKDF2-SHA256 → AES-256-GCM | `lib/crypto-utils.ts` | Encrypt / decrypt in browser |
| **Storage** | Pinata IPFS (pinJSONToIPFS + gateway) | `lib/pinata.functions.ts` | Persist ciphertext |
| **Wallet & RPC** | MetaMask (EIP-1193) + ethers v6 | `lib/wallet.ts, lib/vault-chain.ts` | Sign and send transactions |
| **Contract** | Solidity ^0.8.24, Sepolia testnet | `contracts/DeadMansSwitch.sol` | Custody, timer, release |
| **Automation** | Chainlink Automation interface | `checkUpkeep / performUpkeep` | Trustless trigger |
| **Hosting** | Vite build on Vercel, SPA rewrite | `vercel.json` | Static delivery |

# Trust boundaries: who sees what

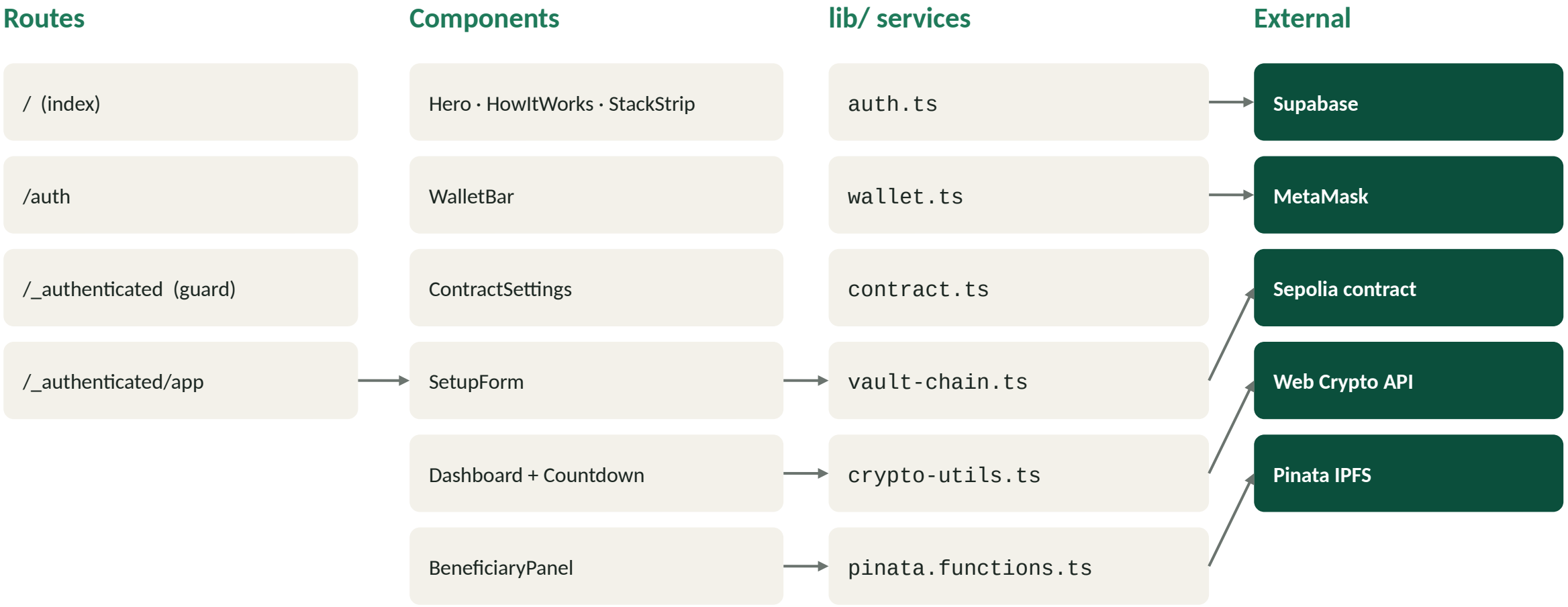
| Component | Plaintext | Passphrase | Ciphertext | CID | Controls ETH |
|---|---|---|---|---|---|
| Owner's / beneficiary's browser tab | Yes | Yes | Yes | Yes | Signs only |
| Pinata / IPFS (public) | — | — | Yes | Yes | — |
| Sepolia contract (public) | — | — | — | Yes | Yes, by rules |
| Supabase database | — | — | — | Yes | — |
| Chainlink Automation | — | — | — | — | Trigger only |



## Zero-knowledge by design

- Only a browser tab ever holds the plaintext or passphrase.
- Everything public (IPFS, chain) is ciphertext or a pointer to it.
- Chainlink can only trigger a release the contract already allows.

# Frontend module map




*Wallet and contract config are tiny external stores read with useSyncExternalStore; the contract address persists in localStorage so one build can target any deployment.*

02

# Smart contract

DeadMansSwitch.sol: one vault per owner, a block-timestamp timer, and a release path anyone can call.

# Vault data model

```
struct Vault {
  address owner;
  address beneficiary;
  uint256 amount;      // locked wei
  uint64  lastPing;    // last heartbeat
  uint64  timeout;     // seconds of silence
  string  cid;         // IPFS ciphertext
  bool released;
  bool cancelled;
  bool exists;
}

mapping(address => Vault) _vaults;
address[] _owners;  // for upkeep scan
```

### One vault per owner address

Keyed by msg.sender. A released or cancelled vault can be re-registered.

### Timer = lastPing + timeout

Due when block.timestamp ≥ deadline. No browser clock is trusted.

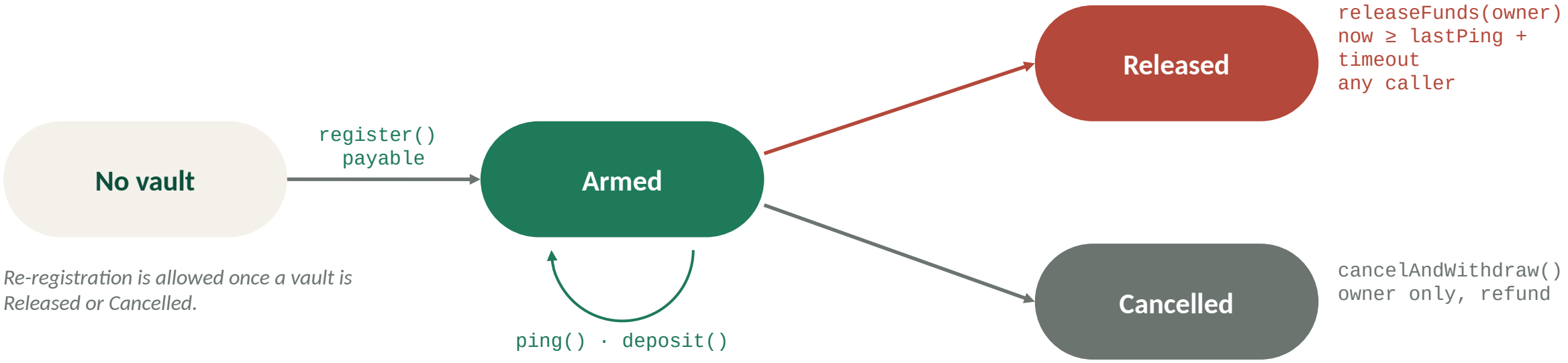
### Bounded intervals

MIN_TIMEOUT = 60 s, MAX_TIMEOUT = 365 days. Beneficiary ≠ 0x0 and ≠ owner.

### Enumerable owners

_owners[] lets checkUpkeep find due vaults on-chain.

# Contract state machine




*Re-registration is allowed once a vault is Released or Cancelled.*

| Function | Caller | Effect |
|---|---|---|
| `register` | anyone | Create vault, lock msg.value |
| `ping` | owner | lastPing = now |
| `deposit` | owner | amount += msg.value |
| `cancelAndWithdraw` | owner | Refund, mark cancelled |
| `releaseFunds` | anyone | Pay beneficiary if due |
| `getVault / isDue` | view | Read state |

# Chainlink Automation: release without a server

### `checkUpkeep()`

Run off-chain by Automation nodes every block. Scans _owners for the first vault past its deadline.

### `performData`

Returns (true, abi.encode(owner)) when a vault is due; (false, "") otherwise.

### `performUpkeep()`

Submitted on-chain by the network. Decodes the owner and calls releaseFunds(owner).

### `releaseFunds()`

Re-checks the deadline, zeroes amount, sets released, pays the beneficiary, emits Released.

**Manual fallback**

releaseFunds is public, so the beneficiary's "Trigger release" button works even if no upkeep is registered.

**Why a permissionless performUpkeep is safe**

Any caller can submit any performData, but releaseFunds reverts with NotDue until the deadline, so nobody can release early.

# Events, errors and safety patterns

## Events (indexed)

- `VaultRegistered(owner, beneficiary, amount, timeout, cid)`
- `Heartbeat(owner, at)`
- `Deposited(owner, amount)`
- `Released(owner, beneficiary, amount)`
- `Cancelled(owner, refunded)`

## Custom errors

- `VaultExists · NoVault · NotOwner`
- `Inactive · BadTimeout`
- `BadBeneficiary · NotDue`
- `TransferFailed`

## Safety patterns

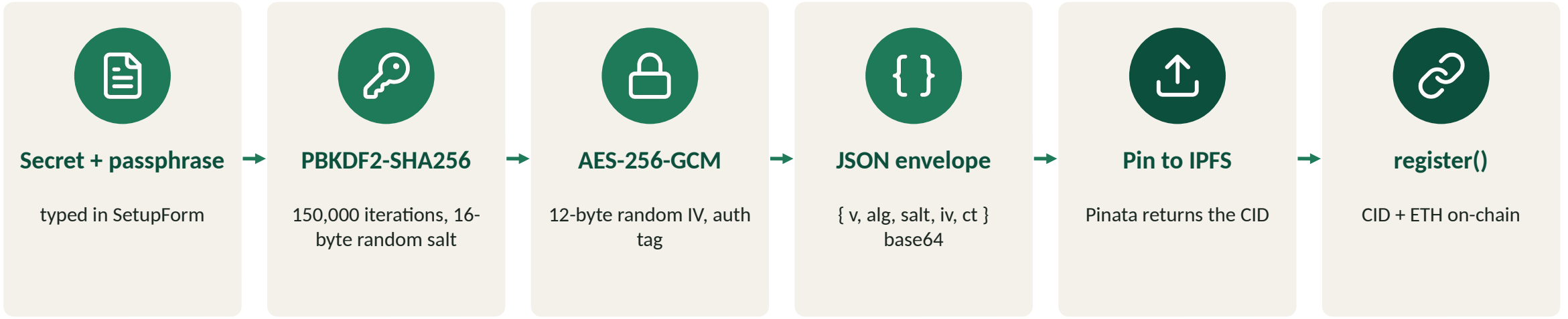
- Checks-effects-interactions: amount zeroed and flags set before the ETH call
- Low-level call with explicit revert on failure
- No admin, owner or upgrade keys
- Custom errors are cheaper than revert strings

03

# Cryptography & storage

The secret is encrypted before it leaves the tab. IPFS stores the ciphertext, the chain stores the CID, and Supabase stores an index.

# Encryption pipeline (owner side)




**150k**
PBKDF2 iterations

**256-bit**
AES-GCM key

**96-bit**
random IV per secret

```
{
  "v": 1,
  "alg": "AES-GCM+PBKDF2",
  "salt": "<base64 16 B>",
  "iv": "<base64 12 B>",
  "ct": "<base64 ciphertext+tag>"
}
```

*Source: src/lib/crypto-utils.ts (encryptSecret / decryptSecret).*

# Decryption pipeline (beneficiary side)

| | | |
|---|---|---|
| 1 | **Read CID** | From the Supabase vault row, also stored on-chain in getVault() |
| 2 | **Fetch ciphertext** | GET gateway.pinata.cloud/ipfs/<cid>; CID format checked with zod |
| 3 | **Validate envelope** | zod schema: v = 1, alg literal, size limits on salt, iv, ct |
| 4 | **Derive key** | PBKDF2 with the stored salt and the passphrase shared out-of-band |
| 5 | **Decrypt + verify** | AES-GCM fails loudly on a wrong passphrase or tampered bytes |



## Authenticated encryption

- GCM's 128-bit tag means a wrong passphrase cannot silently return garbage.
- Any change to salt, IV or ciphertext on IPFS is detected.
- The UI shows one error for both cases, so it leaks nothing about which failed.
- Plaintext is rendered only in the tab and never sent anywhere.

# Off-chain index: Supabase schema

### profiles

```
id  (auth user)
email
display_name
wallet_address
```

### user_roles

```
user_id
role: owner | beneficiary
```

### vaults

```
owner_id · owner_address
beneficiary_address
contract_address · chain_id
ipfs_cid · timeout_seconds
deposit_wei · tx_hash
status · created_at
```

## How the index is used

SetupForm inserts a row after register() confirms → BeneficiaryPanel finds rows for the connected wallet → Each VaultCard re-reads getVault() on-chain

## Why an off-chain index?

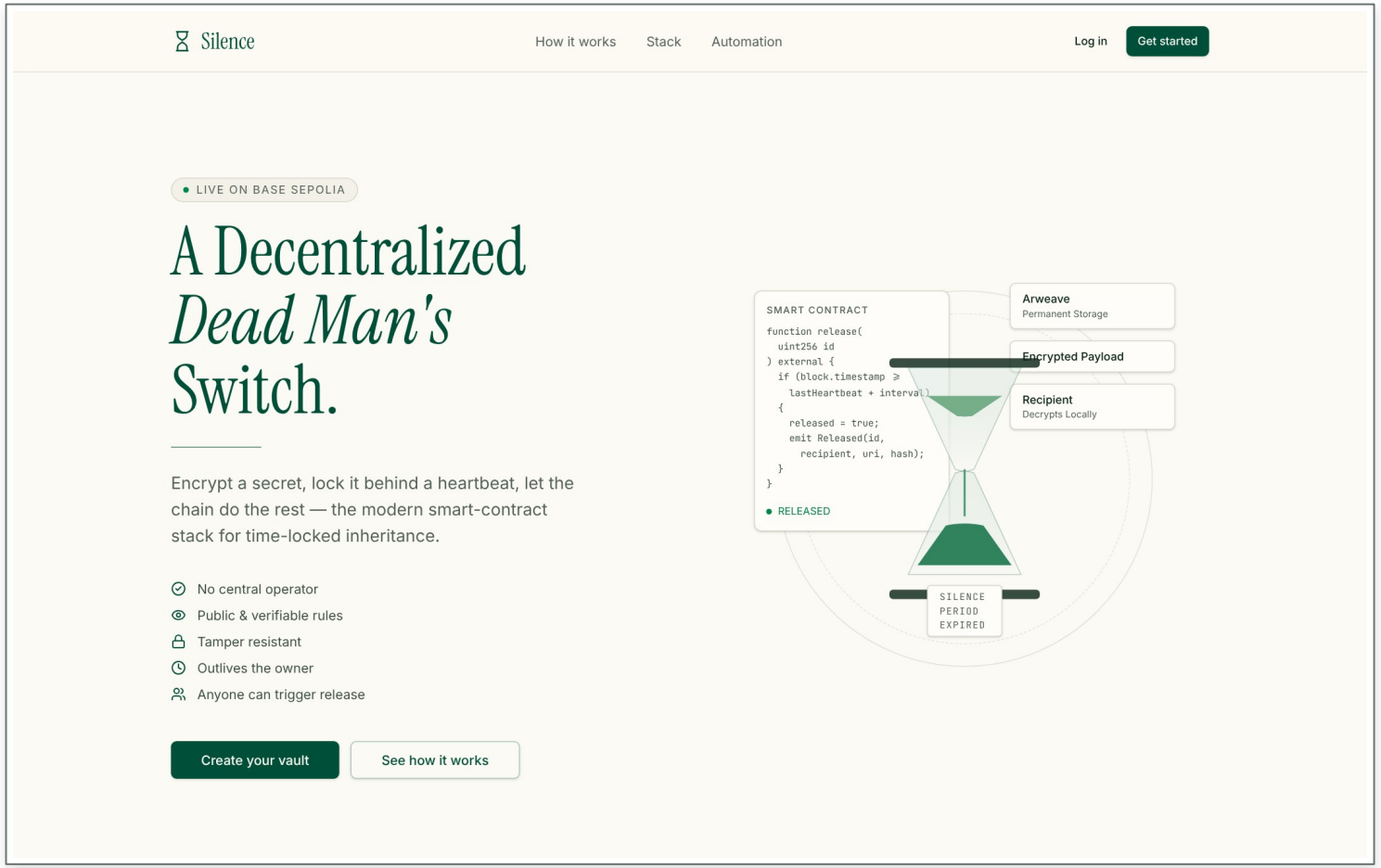
- The contract maps owner → vault only. It has no beneficiary → vaults lookup.
- BeneficiaryPanel queries vaults with ilike(beneficiary_address) to find what is addressed to the connected wallet.
- The chain stays the source of truth: every card re-reads getVault() every 10 s.

04

# User journeys

Screen by screen: landing, sign-in, arming a vault, heartbeats, release, and the beneficiary decrypting.

# Landing page




*Route / · Hero, StackStrip and HowItWorks components*

**Hero** pitches the product with an animated hourglass and contract snippet.

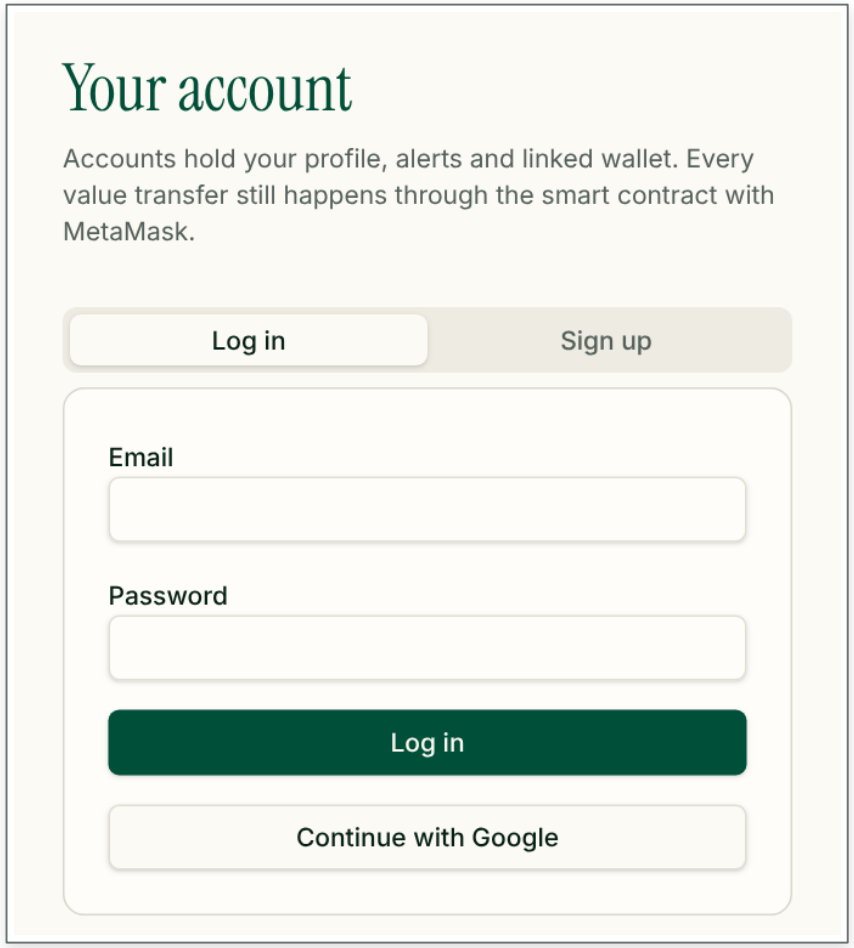
**Lifecycle strip** explains the five states: Register, Heartbeat, Silence, Anyone calls, Released.

**Calls to action** route to /auth, then to the protected /app.




*Lifecycle section further down the page*

# Sign-in, route guard and wallet linking




*Route /auth*

### Supabase Auth

Email/password via signInWithPassword / signUp, or Google OAuth.

### Route guard

/_authenticated runs beforeLoad → supabase.auth.getUser(); no user redirects to /auth.

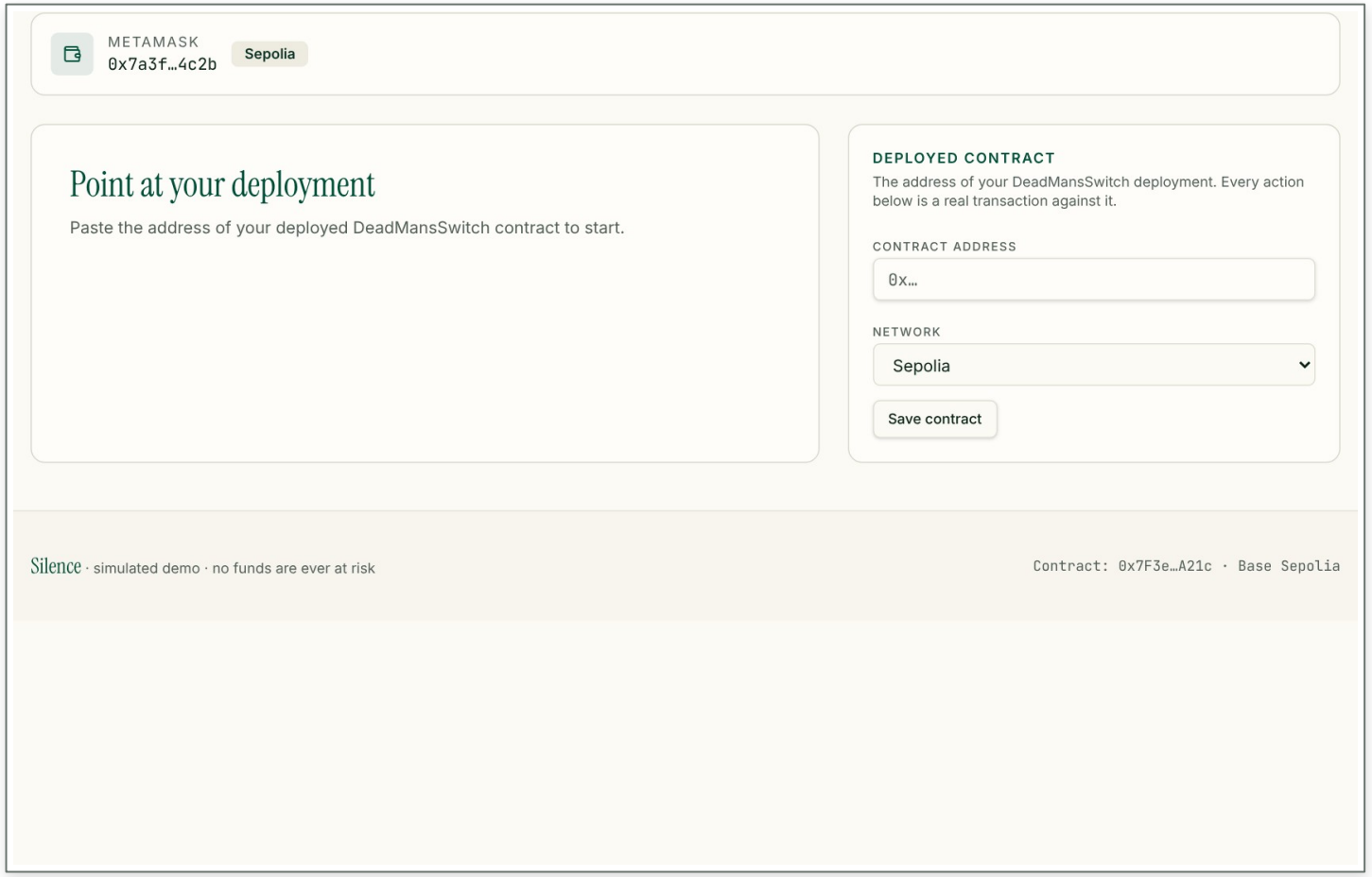
### Connect MetaMask

eth_requestAccounts; account and chain changes are tracked live.

### Link wallet

Address saved to profiles.wallet_address; wrong chain shows a Switch button.

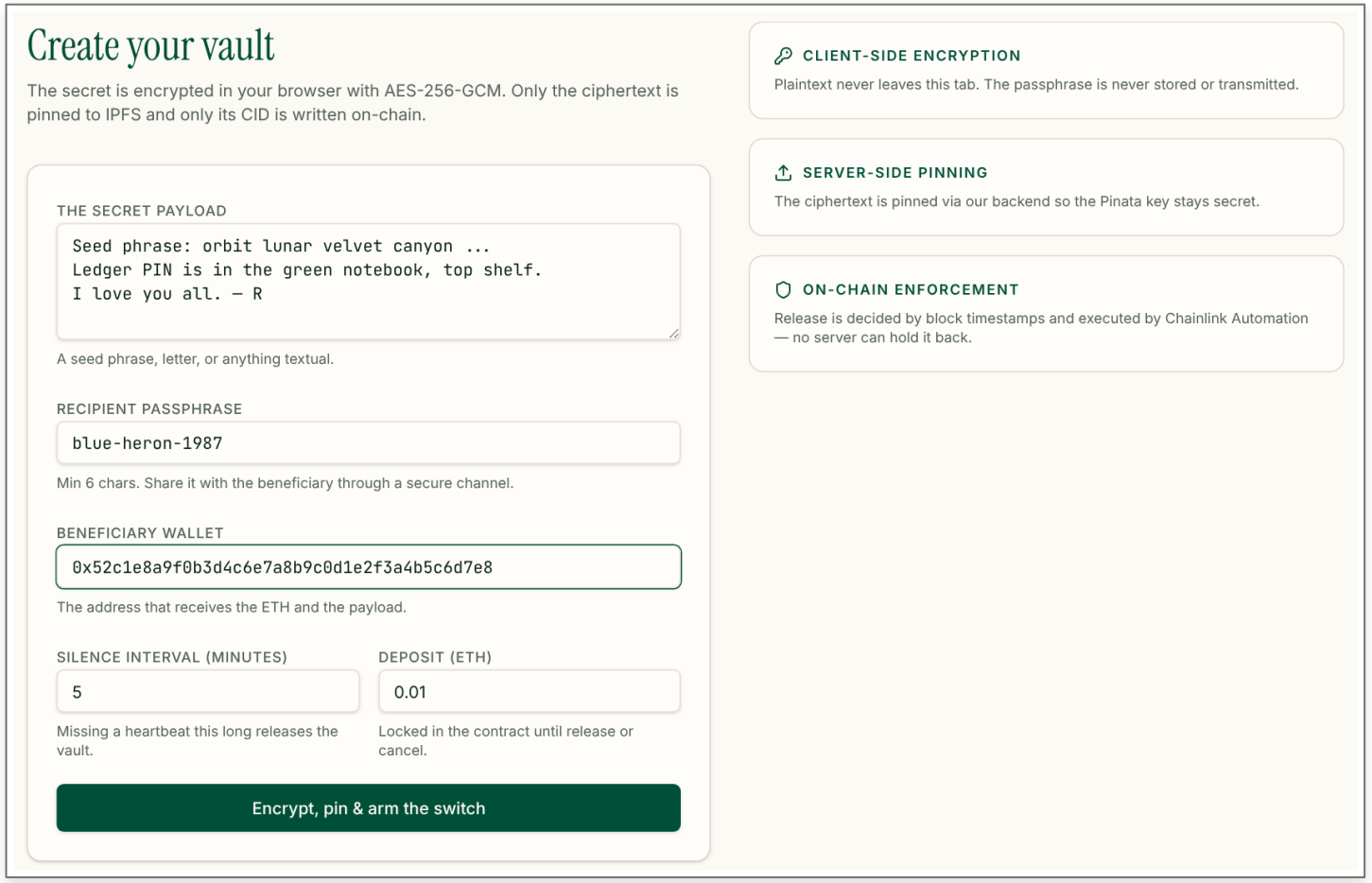
# Pointing the app at a deployment




*Owner view before a contract address is set*

- Address comes from VITE_VAULT_CONTRACT_ADDRESS or is pasted here by the owner.
- Saved to localStorage, so one static build can target any DeadMansSwitch deployment.
- Supported networks: Sepolia (default, chain 11155111), Base Sepolia, Base, Ethereum.
- Explorer links for each transaction are built from the chosen network.

# Owner arms a vault



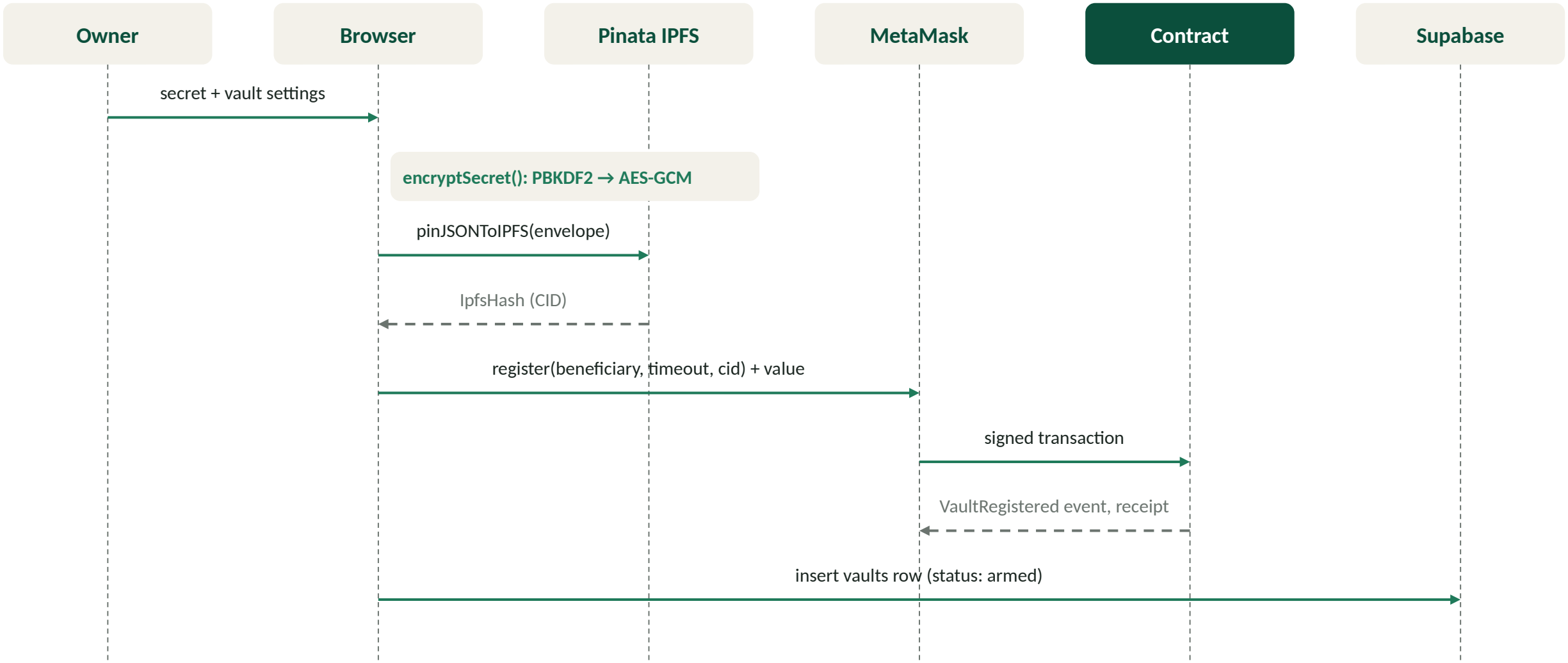

*SetupForm with demo data*

## Button progress states

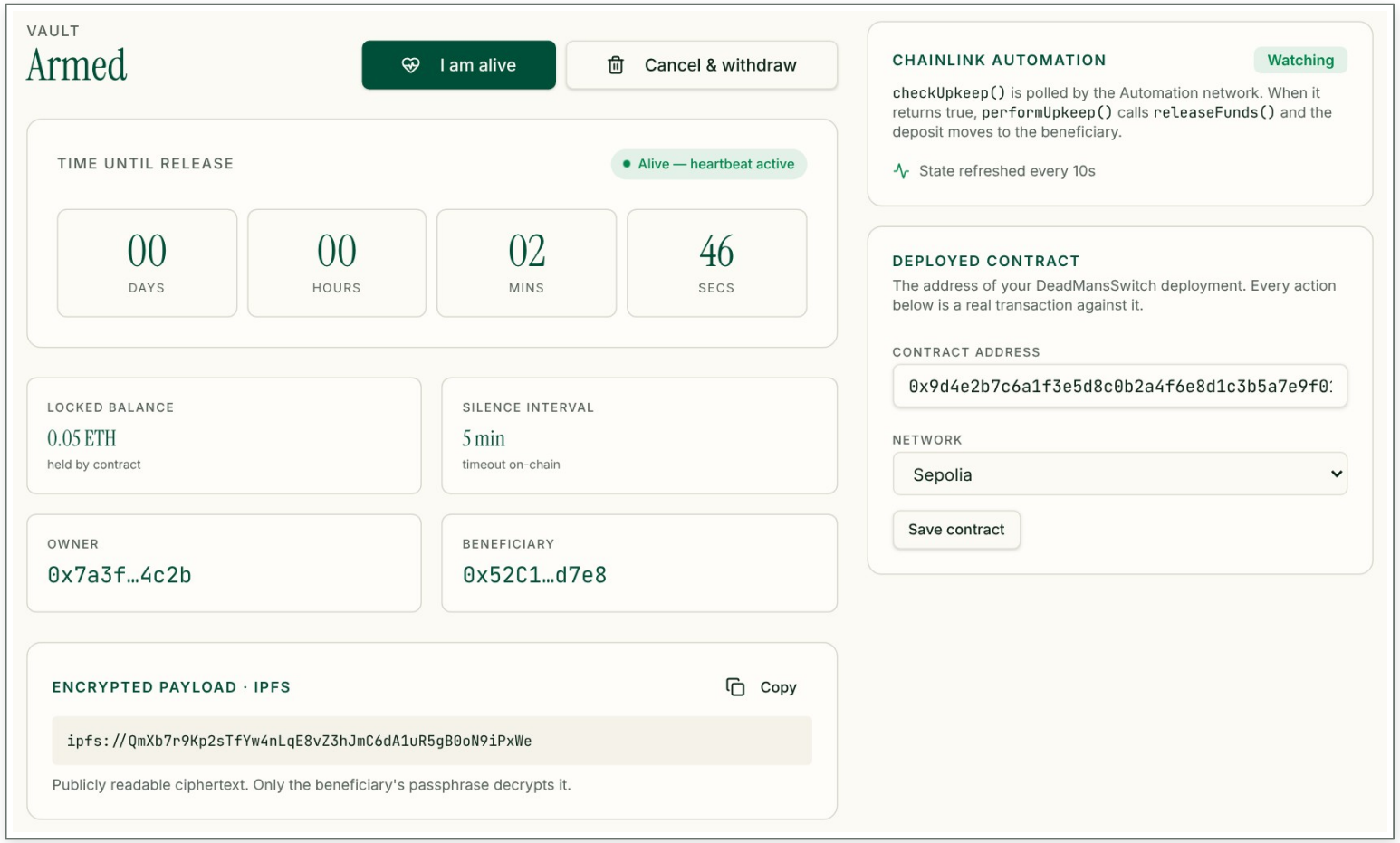
1. Encrypting locally… (AES-256-GCM)
2. Pinning ciphertext to IPFS…
3. Confirm the transaction in MetaMask… register(beneficiary, timeout, cid) with the deposit
4. Saving vault record… (Supabase vaults row)
5. Toast: “Vault armed on-chain”; secret and passphrase cleared from state

*Validation: passphrase ≥ 6 chars, valid beneficiary ≠ owner, interval ≥ 1 min.*

# Sequence: arming a vault end to end



# Owner dashboard: heartbeat and countdown




*Dashboard with an armed vault*

### “I am alive”

Sends ping(); lastPing resets to the block timestamp.

### Countdown

Deadline = lastPing + timeout, read from getVault().

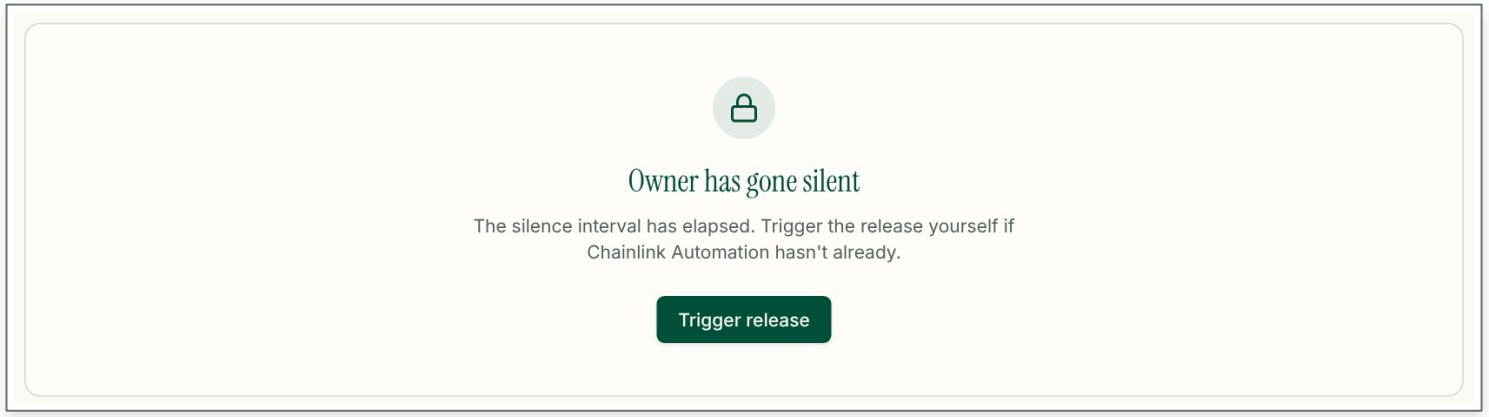
### Cancel & withdraw

cancelAndWithdraw() refunds the deposit and disarms.

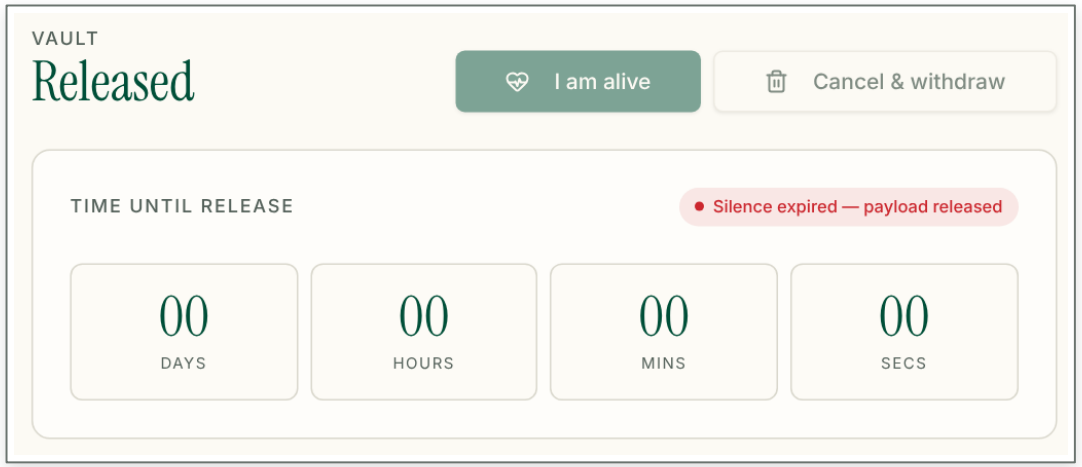
### Live state

TanStack Query refetches the vault every 10 s.

# When the owner goes silent




*Beneficiary view after the interval elapses*




*Owner view after release*

## Two ways to fire

**Automatic:** Chainlink's checkUpkeep spots the due vault and performUpkeep releases it.

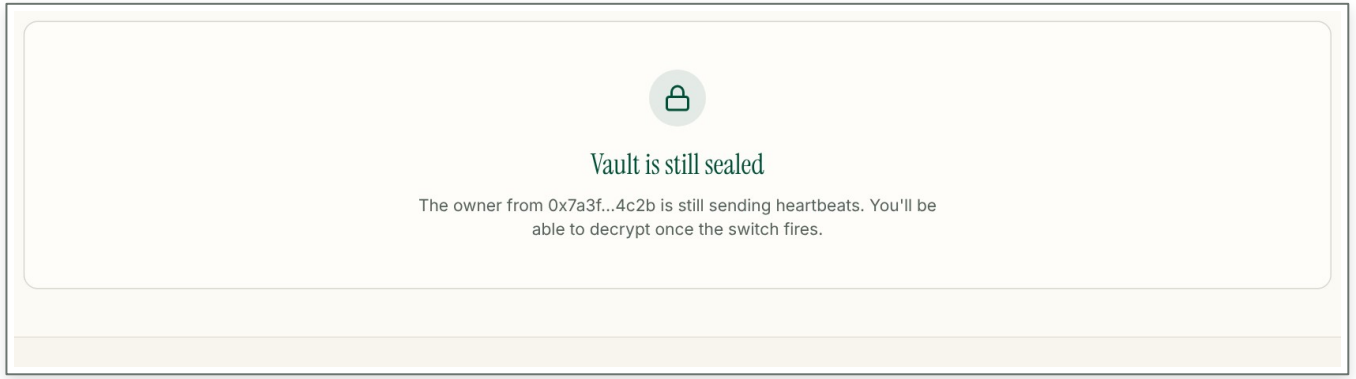
**Manual:** the beneficiary clicks “Trigger release”, which calls releaseFunds(owner).

- Either way, ETH moves to the beneficiary and the vault is marked released.
- The owner's buttons disable and the badge reads “Silence expired”.

# Beneficiary recovers the secret




*Released vault, decrypted in the browser*




*Before release: vault still sealed*

## What happens on Decrypt

- fetchEncryptedPayload({ cid }) reads the envelope from the IPFS gateway.
- decryptSecret(payload, passphrase) derives the key and opens the AES-GCM box.
- The recovered secret is shown only in this tab.
- A wrong passphrase shows a single generic error.

# The full lifecycle at a glance



### Register

`register()`

Encrypt, pin, lock ETH + CID

### Heartbeat

`ping()`

Owner proves liveness

### Silence

`block.timestamp`

Deadline passes unanswered

### Anyone calls

`releaseFunds()`

Chainlink or beneficiary

### Released

`decryptSecret()`

ETH paid, secret decrypted

All timing lives on-chain via block.timestamp, never in a browser timer. The UI countdown is only a view of the contract's deadline.

05

# Review & roadmap

What the design gets right, where the current code falls short, and how to take it further.

# Security review: strengths and known gaps

## Strengths

- Plaintext and passphrase never leave the browser
- Authenticated encryption (AES-GCM) catches tampering
- No admin keys; rules are public and immutable
- Release is permissionless and deadline-checked
- Checks-effects-interactions on every ETH transfer
- Chain is the source of truth; Supabase is only an index

## Known gaps in the current code

**Pinata JWT in the client bundle:** VITE_PINATA_JWT is readable by anyone, although the UI says pinning runs on a backend.

**Weak passphrases:** minimum is 6 chars and the ciphertext is public, so offline guessing is possible.

**checkUpkeep is O(n):** it scans every owner; cost grows with users.

**One vault per address,** one beneficiary per vault.

**Copy drift:** landing mentions Base Sepolia / Arweave; the code uses Sepolia + Pinata.

**No contract tests** are in the repository yet.

# Deployment and configuration

| Variable | Used by | Purpose |
|---|---|---|
| `VITE_SUPABASE_URL` | `integrations/supabase/client.ts` | Supabase project URL |
| `VITE_SUPABASE_PUBLISHABLE_KEY` | `integrations/supabase/client.ts` | Public anon key |
| `VITE_PINATA_JWT` | `lib/pinata.functions.ts` | Pinning auth (move server-side) |
| `VITE_VAULT_CONTRACT_ADDRESS` | `lib/contract.ts` | Default deployment |
| `VITE_VAULT_CHAIN_ID` | `lib/contract.ts` | Default chain, 11155111 |

*Every VITE_ variable is inlined into the public JS bundle at build time.*

### Supported chains (lib/contract.ts)

Sepolia 11155111 (default) · Base Sepolia 84532 · Base 8453 · Ethereum 1



### Deploy contract

scripts/deploy_with_ethers.ts uses a MetaMask signer and ContractFactory.



### Register upkeep

Create a custom-logic upkeep on Chainlink Automation for the address.



### Configure app

Set env vars or paste the address in the UI.



### Ship frontend

vite build on Vercel; vercel.json rewrites all paths to index.html.

# Roadmap

### Server-side pinning

Move Pinata calls into a Supabase Edge Function so the JWT stays secret.

### Public-key encryption

Encrypt to the beneficiary's key (ECIES) instead of a shared passphrase, or use a stronger KDF such as Argon2.

### Richer vaults

Multiple vaults per owner, several beneficiaries, Shamir secret splitting.

### Scalable upkeep

Paginated or per-vault upkeeps instead of scanning every owner.

### Test suite

Foundry tests for every state transition and the timing edge cases.

### Reminders

Email alerts before a deadline so owners don't trigger by accident.

# Silence turns inactivity into a verifiable on-chain event

Encrypt in the browser. Store on IPFS. Let the contract keep time.



### Zero-knowledge

Only ciphertext ever leaves the tab



### Deterministic

block.timestamp decides, not a server



### Unstoppable

Chainlink or anyone can trigger release